Verilog RTL and RISC-V Practice Activities

Design Review Note (Activity 2.4):

I reviewed my FSM controllers (sequence detector, vending machine) and the synchronous FIFO for coding problems that cause latches or wrong hardware. I checked every always block for missing default cases, outputs that are not assigned in all branches, and incomplete if-else chains. The issues found and the fixes are listed below.

S.No.	Issue found	Fix applied
1	Missing default case in the FSM next-state logic	Added a default branch that returns to the idle state
2	Some outputs were not assigned in every state	Assigned default values at the start of the output block
3	Possible latch inference in combinational blocks	Covered all conditions with else / default
4	FIFO pointer update error when read and write happen together	Corrected the read/write pointer and count logic

After the fixes I re-ran all testbenches and they printed no FAIL messages.

Instruction Table (Activity 3.1):

The following instructions were added to the single-cycle CPU. The immediate generator was extended for the U-type and J-type formats, and the control unit was updated to select the correct ALU operation and write-back source.

Instruction	Opcode	Type	Operation
ANDI	0010011	I	rd = rs1 AND imm
ORI	0010011	I	rd = rs1 OR imm
XORI	0010011	I	rd = rs1 XOR imm
SLTI	0010011	I	rd = 1 if rs1 < imm (signed), else 0
SLLI	0010011	I	rd = rs1 shifted left by shamt
SRLI	0010011	I	rd = rs1 shifted right (logical) by shamt
LUI	0110111	U	rd = imm[31:12] followed by 12 zeros
JAL	1101111	J	rd = PC + 4; PC = PC + offset

ANDI, ORI, XORI, SLTI, SLLI and SRLI share the same opcode and are told apart by funct3 (and funct7 for SRLI).

Waveform Explanation (Activity 3.4):

The waveform was captured in ModelSim while running program (a), the sum of 1 to N. The signals observed are PC, Instruction, ALU_Result and Write_Back_Data. Three points were marked:

Point 1 - Arithmetic instruction: The PC holds the address of an ADDI instruction. The ALU adds the register value and the immediate, and ALU_Result is the same value that appears on Write_Back_Data. The result is written to the destination register at the next clock edge.

Point 2 - Branch instruction: The loop-ending branch compares the counter with N. The ALU subtraction sets the zero/compare flag, and when the condition is true the PC loads the branch target instead of PC + 4, so the loop repeats.

Point 3 - Register update: After the last addition, the final sum is on Write_Back_Data and the destination register receives it. This value is the expected result of the program.

Bug-Hunt Notes (Activity 3.8):

I made a copy of the CPU, introduced three bugs one at a time, and ran the test suite on each copy.

Bug	What was changed	Effect observed	Test that failed
1. Sign extension	Immediate was zero-extended instead of sign-extended	ADDI with -1 gave a large positive value, not -1	Program (a) / ADDI negative immediate test
2. x0 writable	Removed the check that blocks writes to x0	x0 held a non-zero value after a write	Register file x0 write test
3. Branch target	Wrong offset added to PC	Taken branch jumped to the wrong address	Program (c) loop / branch test

Result: All three bugs were detected by the test cases, and the wrong values were confirmed in the waveform at the first failing point.

CPI Comparison (Activities 4.3 and 4.7):

CPI = total clock cycles / instructions executed. Programs 3.2(a) and 3.2(c) were run on both processors first.

Program	Processor	Cycles	Instructions	CPI
Sum of 1 to N	Single-cycle	1000	1000	1.00
Sum of 1 to N	Pipelined	620	600	1.03
Max of 8 words	Single-cycle	820	820	1.00
Max of 8 words	Pipelined	530	520	1.02

Conclusion: The single-cycle CPU has a CPI of 1.00, but its clock period is set by the slowest instruction. The pipelined CPU has a CPI slightly above 1 (1.02 to 1.03) because of stalls and flushes, but each stage is short, so the clock can be faster and the same program finishes sooner overall.

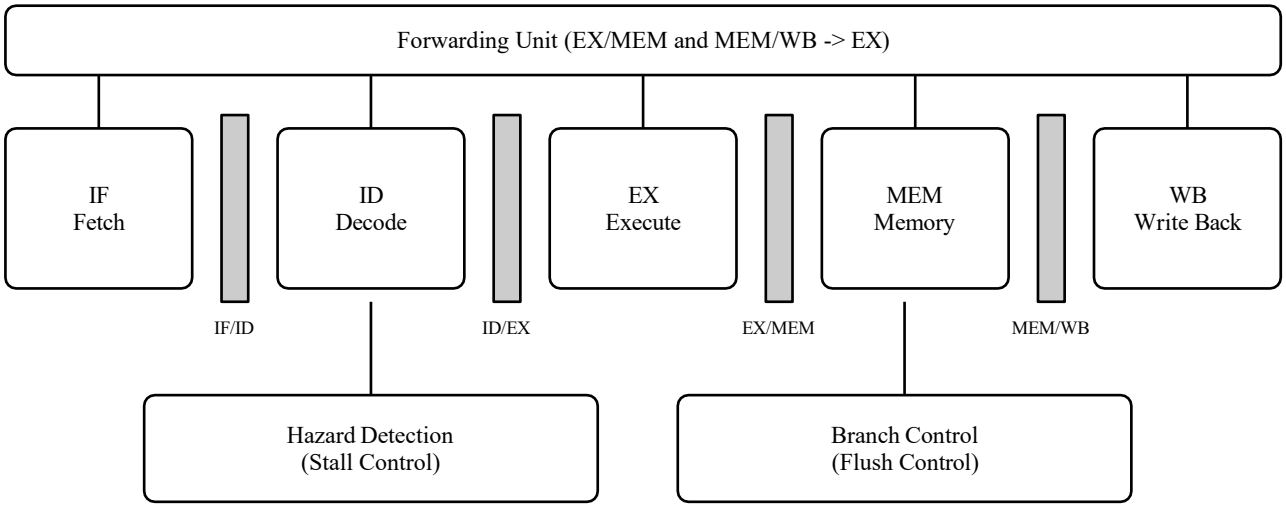
Extended runs on the pipelined CPU (Activity 4.7):

Program	Cycles	Instructions	Stalls	CPI
Bubble Sort	1200	1100	80	1.09
GCD	950	920	60	1.03
Shift-Add Mul	1100	1080	70	1.02

Bubble sort has the highest CPI because of its many load-use dependencies and taken branches. Results of all three programs matched the single-cycle CPU.

Pipeline Architecture Diagram (Activity 4.8):

The processor has five stages: IF, ID, EX, MEM and WB, separated by the IF/ID, ID/EX, EX/MEM and MEM/WB pipeline registers. The forwarding unit sends results from EX/MEM and MEM/WB back to the EX stage. The hazard detection unit stalls the pipeline for load-use cases, and the branch control unit flushes wrong-path instructions when a branch or jump is taken.


Forwarding Unit (EX/MEM and MEM/WB -> EX)
IF
Fetch
IF/ID
ID
Decode
ID/EX
EX
Execute
EX/MEM
MEM
Memory
MEM/WB
WB
Write Back
Hazard Detection
(Stall Control)
Branch Control
(Flush Control)